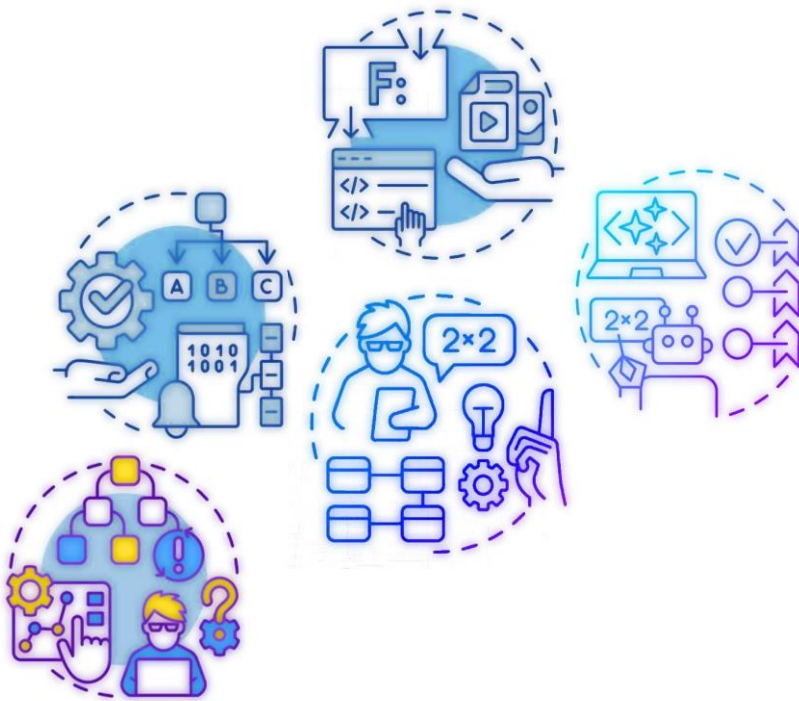




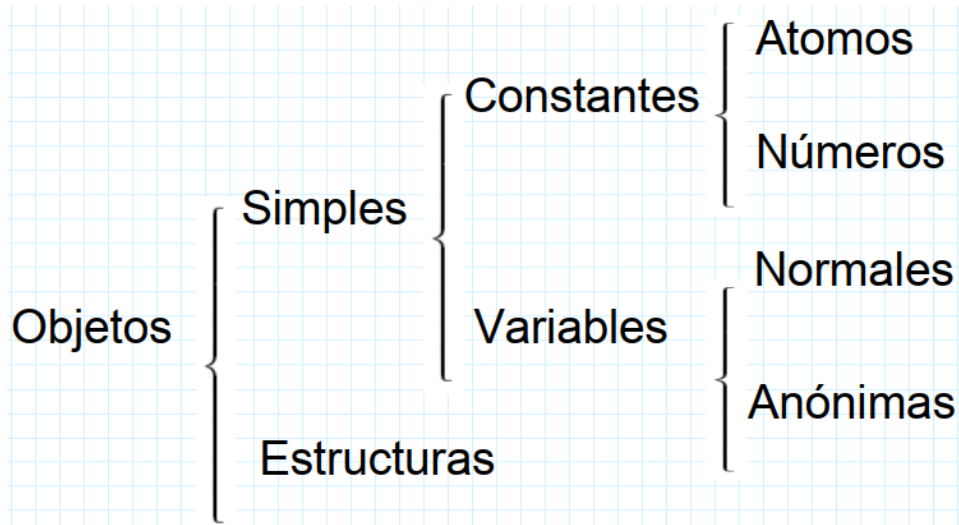
Paradigma Lógico

Esp. Ing. Viviana A. Santucci
AIA Federico Castoldi
Ing. J. Exequiel Benavidez
Ing. Jimena Bourlot

Tecnologías de la Programación

1 – ELEMENTOS DEL LENGUAJE

La forma correcta y más fundamental de definir los elementos para describir la sintaxis y los tipos de términos básicos que se usan para escribir programas en Prolog es la que se presenta en la imagen:



La imagen describe la sintaxis y los tipos de términos básicos que se usan para escribir programas en Prolog.

Son los "bloques de construcción" básicos con los que se expresa el conocimiento

1 – ELEMENTOS DEL LENGUAJE - Objetos Simples

Constantes

Representan objetos o relaciones específicos e inmutables.

Hay dos tipos de constantes:

- Átomos: Nombres simbólicos (ej. juan, maria, 'Lunes').
- Números: Valores numéricos (ej. 123, 3.14).

1 – ELEMENTOS DEL LENGUAJE - Objetos Simples

Constantes

- Átomos:

Ejemplos de átomos son: likes mary john book wine owns jewels
can_steal

Los símbolos especiales que Prolog usa para denotar preguntas '?-'
, y reglas ':-', también son átomos.

Hay dos tipos de átomos: aquellos formados por letras y dígitos, y
aquellos formados por signos.

1 – ELEMENTOS DEL LENGUAJE - Objetos Simples

Constantes

El primer tipo normalmente debe comenzar con una letra minúscula, Esos átomos hechos de signos normalmente se componen solo de signos.

A veces puede ser necesario tener un átomo que comience con una letra mayúscula o un dígito.

Si un átomo está encerrado entre comillas simples "'", entonces el átomo puede tener cualquier carácter en su nombre.

Finalmente, el carácter de subrayado '_' puede insertarse en medio de un átomo para mejorar la legibilidad.

1 – ELEMENTOS DEL LENGUAJE - Objetos Simples

Constantes

Los siguientes son ejemplos adicionales de átomos:

`a void = 'george-smith' --> george_smith ieh2304`

Los siguientes no son ejemplos de átomos:

`2304ieh george-smith Void _alpha`

1 – ELEMENTOS DEL LENGUAJE - Objetos Simples

Constantes

- Números:

Los números son el otro tipo de constante. Ejemplos de números:

-17 -2.67e2 0 1 99.9 512 8192 14765 67344 6.02e-23

La mayoría de estos son familiares. La notación 'e' se usa para denotar una potencia de 10.

Así, por ejemplo, el número -2.67e2 es -2.67×10^2 o simplemente -267; 6.02e-23 es 6.02×10^{-23} . Además de estos números, los programadores de Prolog han añadido librerías para definir características como operaciones aritméticas con números racionales y números de precisión arbitraria.

1 – ELEMENTOS DEL LENGUAJE - Objetos Simples Variables

Variables: Representan objetos que aún no están especificados o que pueden variar.

- Normales: Variables con nombre (ej. X, Y, Nombre). Nombres que empiezan con mayúscula o guión bajo (ej. X, Y, AlgunMas).
- Anónimas: Variables cuyo valor no nos importa (ej. _). Empiezan con Un solo guión bajo (_).

1 – ELEMENTOS DEL LENGUAJE - Objetos Simples Variables

Variables normales: se parecen a los átomos, excepto que sus nombres comienzan con una letra mayúscula o un signo de subrayado '_'.

Una variable debe ser considerada como representando algún objeto que no podemos o no queremos nombrar en el momento en que escribimos el programa.

En las cláusulas Prolog de ejemplo hemos usado variables con nombres como X, Y y Z. Sin embargo, los nombres pueden ser tan largos como se desee, por ejemplo:

Answer Input Gross_Pay _3_blind_mice A_very_long_variable_name

1 – ELEMENTOS DEL LENGUAJE - Objetos Simples Variables

Variables anónimas: A veces es necesario usar una variable, pero su nombre nunca será utilizado. Por ejemplo, si queremos saber si a alguien le gusta Juan, pero no necesitamos saber quién es exactamente, podemos usar la variable anónima. La variable anónima se escribe como un único carácter de subrayado. Nuestro ejemplo se escribe en Prolog como:

?- likes(_, john).

Varias variables anónimas en la misma cláusula no necesitan tener interpretaciones consistentes. Esta es una característica peculiar de la variable anónima. Se utiliza para evitar tener que inventar diferentes nombres de variables cuando no se van a utilizar en otra parte de la cláusula."

1 – ELEMENTOS DEL LENGUAJE - Objetos Estructuras

Una estructura es un objeto único que consta de una colección de otros objetos, llamados componentes. Los componentes se agrupan en una única estructura para facilitar su manejo.

Las estructuras ayudan a organizar los datos en un programa porque permiten que un grupo de información relacionada sea tratado como un objeto único (una ficha de biblioteca) en lugar de como entidades separadas. La forma en que se descomponen los datos en componentes depende del problema que se quiera resolver

1 – ELEMENTOS DEL LENGUAJE - Objetos Estructuras

Una estructura se escribe en Prolog especificando su functor (cuyo primer carácter debe ser una letra minúscula y argumentos que pueden ser constantes, variables u otros funtores) y sus componentes.

El functor nombra el tipo general de estructura, y corresponde a un tipo de dato en un lenguaje de programación ordinario.

Los componentes se encierran entre paréntesis y se separan por comas. El functor se escribe justo antes del paréntesis de apertura.

1 – ELEMENTOS DEL LENGUAJE - Objetos Estructuras

Considera el siguiente hecho, que Juan posee el libro llamado Cumbres Borrascosas, de Emily Brontë:

`owns(john, book(wuthering_heights, bronte)).`

Dentro del hecho `owns`, tenemos una estructura llamada `book`, que tiene dos componentes: un título y un autor. Dado que la estructura `book` aparece dentro del hecho como uno de los argumentos del hecho, está actuando como un objeto, participando en una relación.

1 – ELEMENTOS DEL LENGUAJE - Objetos Estructuras

Si lo deseamos, también podemos tener otra estructura para el nombre del autor, porque hay tres escritoras Brontë que queremos distinguir:

owns(john, book(wuthering_heights, author(emily, bronte))).

Las estructuras pueden participar en el proceso de respuesta a preguntas utilizando variables. Por ejemplo, podemos preguntar si Juan posee algún libro de alguna de las hermanas Brontë:

?- owns(john, book(X, author(Y, bronte))).

1 – ELEMENTOS DEL LENGUAJE - Distinción

Los “**elementos**” del lenguaje Prolog a un nivel fundamental y estructural son las piezas.

Los “**hechos, reglas y preguntas**” son los tipos de sentencias o construcciones lógicas que se pueden formar con esos elementos del lenguaje, y son la forma en que el programador interactúa con el sistema Prolog para definir conocimiento y hacer consultas.

1 – ELEMENTOS DEL LENGUAJE - Distinción

Es decir, una cosa son las piezas y otra cosa son las formas en que esas piezas se ensamblan para crear significado (hechos, reglas, preguntas). Ambas son correctas, pero en diferentes niveles de abstracción. La imagen con la jerarquía de "Elementos del Lenguaje" es la forma correcta y más fundamental de definir los componentes básicos de Prolog.

Mientras que "hechos, reglas y preguntas" son categorías de cláusulas o sentencias en Prolog, y describen cómo se organiza la base de conocimiento y cómo interactuamos con ella.

2 - SIGNIFICADO DECLARATIVO Y PROCEDURAL

En un lenguaje declarativo puro, sería de esperar que el orden en el que aparecen los hechos y las reglas en la base fuera independiente de los datos, sin embargo en PROLOG no es así:

Los cálculos y unificaciones se realizan en forma procedural, por lo que el orden de los predicados, y en que se declaran las proposiciones de los mismos, influyen en el desarrollo

$X=4, Y=2, X>Y$. no es lo mismo que escribir $X=4, X>Y, Y=2$.

En particular el segundo caso falla, ¿por qué?

2 - SIGNIFICADO DECLARATIVO Y PROCEDURAL

A pesar de que Prolog se basa en la lógica de primer orden (que es declarativa por definición), su estrategia de ejecución (resolución SLD) es inherentemente procedural y esto introduce una dependencia del orden.

Caso 1: $X=4$, $Y=2$, $X>Y$.

$X=4$: Prolog unifica la variable X con el valor 4. X ahora está ligada a 4.

$Y=2$: Prolog unifica la variable Y con el valor 2. Y ahora está ligada a 2.

$X>Y$: Prolog evalúa la relación $>$ (mayor que). Como X es 4 e Y es 2, la comparación $4 > 2$ es verdadera.

Resultado: La consulta tiene éxito y devuelve true.

2 - SIGNIFICADO DECLARATIVO Y PROCEDURAL

Caso 2: $X=4$, $X>Y$, $Y=2$.

$X=4$: Prolog unifica la variable X con el valor 4. X ahora está ligada a 4.

$X>Y$: Prolog intenta evaluar la relación $>$ (mayor que).

En este punto, X está ligada a 4.

Sin embargo, Y es una variable libre (no está ligada a ningún valor).

2 - SIGNIFICADO DECLARATIVO Y PROCEDURAL

El predicado $>$ (y la mayoría de los predicados aritméticos de comparación como $<$, $\geq$, $\leq$, $=$, $\neq$) requieren que ambos argumentos sean valores numéricos instanciados (no variables libres) para poder realizar la comparación.

Como Y no tiene un valor numérico en este momento, la comparación $4 > Y$ no puede ser evaluada.

Resultado: El predicado $X > Y$ falla porque sus argumentos no están suficientemente instanciados.

2 - SIGNIFICADO DECLARATIVO Y PROCEDURAL

El predicado $>$ (y la mayoría de los predicados aritméticos de comparación como $<$, $>=$, $=<$, $:=$, $=\backslash=$) requieren que ambos argumentos sean valores numéricos instanciados (no variables libres) para poder realizar la comparación.

Como Y no tiene un valor numérico en este momento, la comparación $4 > Y$ no puede ser evaluada.

Resultado: El predicado $X > Y$ falla porque sus argumentos no están suficientemente instanciados.

2 - SIGNIFICADO DECLARATIVO Y PROCEDURAL

¿Por qué el segundo caso falla?

El segundo caso falla precisamente por la estrategia de evaluación de Prolog: de izquierda a derecha y de arriba hacia abajo (depth-first search).

Prolog procesa los objetivos (predicados) en el cuerpo de una cláusula o en una consulta secuencialmente, de izquierda a derecha.

Cuando llega a $X > Y$, necesita los valores concretos de X e Y para poder ejecutar esa operación de comparación.

En el segundo caso, Y aún no ha sido instanciada ($Y=2$ está a la derecha de $X > Y$), por lo que el predicado $X > Y$ no puede proceder y falla.

2 - SIGNIFICADO DECLARATIVO Y PROCEDURAL

Esto resalta la distinción entre la naturaleza declarativa de la lógica (donde el orden de las proposiciones no importa) y la naturaleza procedural (ejecutable) de Prolog.

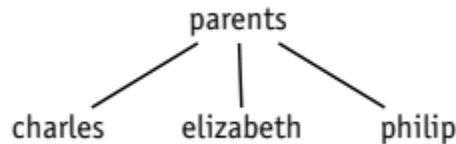
Si bien Prolog se esfuerza por ser declarativo, su motor de inferencia sigue un algoritmo paso a paso que impone un orden de ejecución y, por lo tanto, una dependencia en la instanciación de las variables en el momento adecuado.

Es uno de los aspectos más importantes a tener en cuenta al diseñar programas en Prolog.

2 - ÁRBOLES

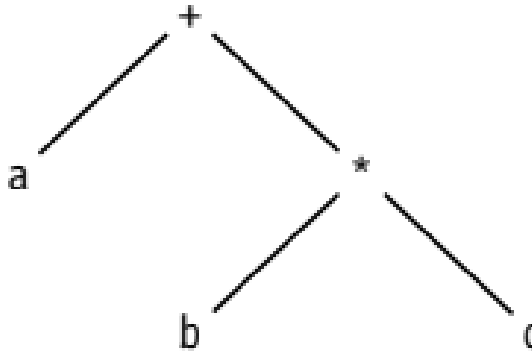
Suele ser más fácil entender la forma de una estructura complicada si la escribimos como un árbol, en el que cada functor es un nodo y los componentes son ramas.

Cada rama puede apuntar a otra estructura, de modo que podemos tener estructuras dentro de estructuras. Es costumbre dibujar un diagrama de árbol con la raíz en la parte superior y las ramas en la parte inferior. Por ejemplo, la estructura `parents(charles, elizabeth, philip)` se escribe como:"



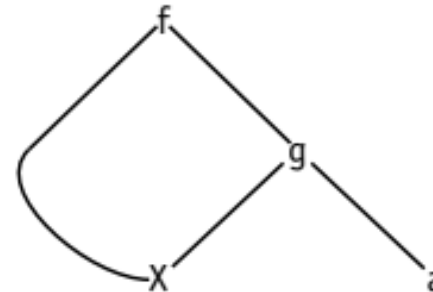
2 - ÁRBOLES

La estructura $a+b*c$ (o, equivalentemente, $+(a, *(b,c))$) se escribe como:



2 - ÁRBOLES

Los árboles también pueden dar una descripción gráfica de las variables dentro de las estructuras, en particular mostrando cómo se comparten las variables con nombres similares. Por ejemplo, podemos representar la estructura del término $f(X, g(X, a))$ mediante el siguiente árbol (más precisamente, un grafo acíclico dirigido):"



2 - LISTAS

Es una estructura de datos simple que denota una secuencia de elementos tales como: 1, 2, 3, 4, 5

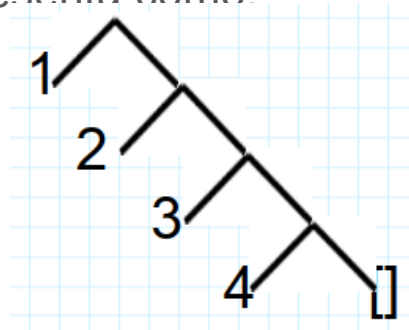
La sintaxis para denotar una lista es encerrar los elementos separados por coma entre corchetes:

[1, 2, 3, 4, 5]

Internamente se representa mediante un árbol binario donde la rama de la derecha representa el primer elemento o cabeza, y la rama de la izquierda representa el resto o cola de la lista

2 - LISTAS

Así, la lista [1, 2, 3, 4] se representa como:



donde la lista [] es la lista vacía.

Aunque no es necesario, ProLog representa las listas a través del functor “.”

Así [1, 2, 3, 4] es equivalente a `.(1, .(2, .(3, .(4, []))))`

2 - LISTAS

La cabeza y la cola se pueden separar mediante el símbolo “|”, así las siguientes expresiones se refieren a la misma lista:

- **[a,b,c]**
- **[a|[b,c]]**
- **[a,b|[c]]**
- **[a,b,c|[]]**

2 - LISTAS

Ejemplo de uso:

Clásico de recursividad en Prolog para determinar si un elemento es miembro de una lista. Se basa en la descomposición de la lista en su "cabeza" y su "cola".

```
miembro(X,[X|_]).
```

```
miembro(X,[_|R]):-miembro(X,R).
```

2 - LISTAS

`miembro(X,[X|_]).` (Cláusula base)

Lectura: X es miembro de una lista si X es el primer elemento (la "cabeza") de esa lista.

Significado: Esta es la condición de parada o caso base. Si el elemento que buscamos (X) es el mismo que el primer elemento de la lista, entonces hemos terminado y es verdadero. _ (la variable anónima) indica que no nos importa el resto de la lista en este caso.

2 - LISTAS

miembro(X,[_|R]):-miembro(X,R). (Cláusula recursiva)

Lectura: X es miembro de una lista si X no es el primer elemento de la lista (indicado por _), Y X es miembro del resto de la lista (la "cola", R).

Significado: Esta es la definición recursiva. Si el elemento no está al principio, "descartamos" el primer elemento y llamamos a miembro nuevamente con la misma X pero con una lista más corta (R). Esto se repite hasta que se encuentre el elemento (caso base) o la lista se quede vacía y la búsqueda falle.

2 - LISTAS

Ejemplo de uso:

Concatenar dos listas:

`concatenar ([],L,L).`

`concatenar ([X|L1],L2,[X|L3]):- concatenar (L1,L2,L3).`

2 - LISTAS

concatenar([], L, L). (Cláusula base)

Lectura: Concatenar la lista vacía ([]) con cualquier lista L da como resultado la misma lista L.

Significado: Este es el caso base. Si la primera lista está vacía, no hay nada que añadir, por lo que la lista resultante es simplemente la segunda lista.

2 - LISTAS

concatenar([X|L1], L2, [X|L3]) :- concatenar(L1, L2, L3). (Cláusula recursiva)

Lectura: Para concatenar una lista que tiene una cabeza X y una cola L1 con una segunda lista L2, el resultado será una lista cuya cabeza es X (la misma cabeza de la primera lista), Y cuya cola L3 es el resultado de concatenar el resto de la primera lista (L1) con la segunda lista completa (L2).

Significado: Esta es la definición recursiva. Toma el primer elemento (X) de la primera lista y lo "pone" como el primer elemento de la lista resultado. Luego, llama recursivamente a concatenar para unir el resto de la primera lista (L1) con la segunda lista (L2), y el resultado de esa llamada recursiva se convierte en la cola (L3) de la lista final.

2 - LISTAS

En resumen:

El predicado concatenar(Lista1, Lista2, Resultado) funciona "desarmando" la Lista1 elemento por elemento. Cada elemento que saca de Lista1 lo va "poniendo" en la Resultado final, hasta que la Lista1 se vacía. En ese momento, la Lista2 completa se "pega" al final de lo que se ha construido, formando la Resultado final.

2 - LISTAS

Ejemplo de uso:

Dado una lista calcular su inversa:

```
inversa([],[]).
```

```
inversa([X|L1],L):-
```

```
    inversa(L1,Resto),
```

```
    concatenar(Resto,[X],L)
```

3 - CONSTRUCCIÓN DE EXPRESIONES ARITMÉTICAS

- $X + Y$
- $X - Y$
- $X * Y$
- X / Y o $X \text{ div } Y$
- $X \bmod Y$
- X^Y
- $-X$
- $\text{abs}(X)$
- $\text{acos}(X)$
- $\text{asen}(X)$
- $\text{atan}(X)$
- $\text{cos}(X)$
- $\text{exp}(X)$
- $\text{ln}(X)$
- $\text{log}(X)$
- $\text{sin}(X)$
- $\text{sqrt}(X)$
- $\text{tan}(X)$
- $\text{round}(X, N)$

4 - COMPARACIÓN DE TÉRMINOS

- $X < Y$ /*X es menor que Y*/
- $X > Y$ /*X es mayor que Y*/
- $X \leq Y$ /*X es menos o igual que Y*/
- $X \geq Y$ /*X es mayor o igual que Y*/
- $X = Y$ /*X es igual que Y*/
- $X \neq Y$ /*X es distinto que Y*/

5 - EXPRESIONES (COMPARACIÓN)

Una expresión es un conjunto de términos unidos por operadores aritméticos. Los siguientes operadores comparan expresiones en forma sintáctica:

- $X == Y$ /*la expresión X es igual que la expresión Y*/
- $X \neq Y$ /*la expresión X es distinta que la expresión Y*/
- $X < Y$ /*la expresión X es menor que la expresión Y*/
- $X > Y$ /*la expresión X es mayor que la expresión Y*/
- $X \leq Y$ /*la expresión X es menor o igual que la expresión Y*/
- $X \geq Y$ /*la expresión X es mayor o igual que la expresión Y*/

5 - EXPRESIONES (EVALUACIÓN)

El siguiente operador permite asignar la evaluación de una expresión

- $X \text{ is } Y$
- Ej.: $X \text{ is } (3*5+3)/2$ /*PROLOG contestaría $X=9$ */

Los siguientes operadores comparan la evaluación de dos expresiones

- $X := Y$ /* El resultado de evaluar la expresión X es igual al resultado de evaluar la expresión Y */
- $X \neq Y$ /* El resultado de evaluar la expresión X es distinto al resultado de evaluar la expresión Y */

5 - EXPRESIONES (EVALUACIÓN)

Ejemplo:

Encontrar la cantidad de números positivos y negativos de una lista:

positivo(X) :- X >= 0.

negativo(X) :- X < 0.

evaluar([], 0, 0).

evaluar([X|Lista], Pos, Neg) :-

positivo(X),
evaluar(Lista, Pos1, Neg),
Pos is Pos1 + 1.

evaluar([X|Lista], Pos, Neg) :-

negativo(X),
evaluar(Lista, Pos, Neg1),
Neg is Neg1 + 1.